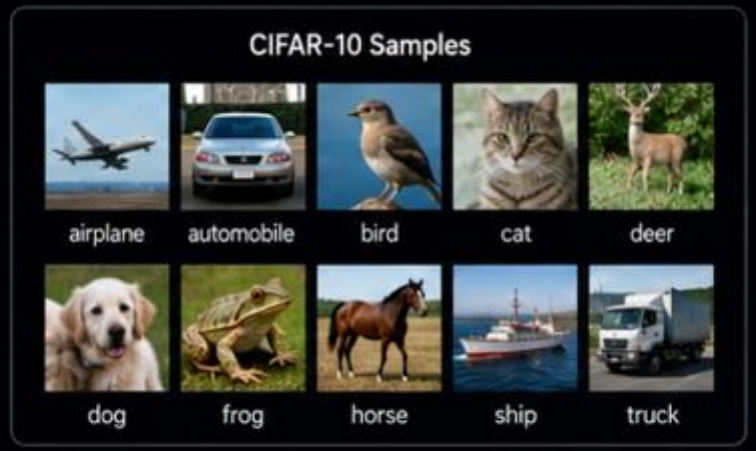
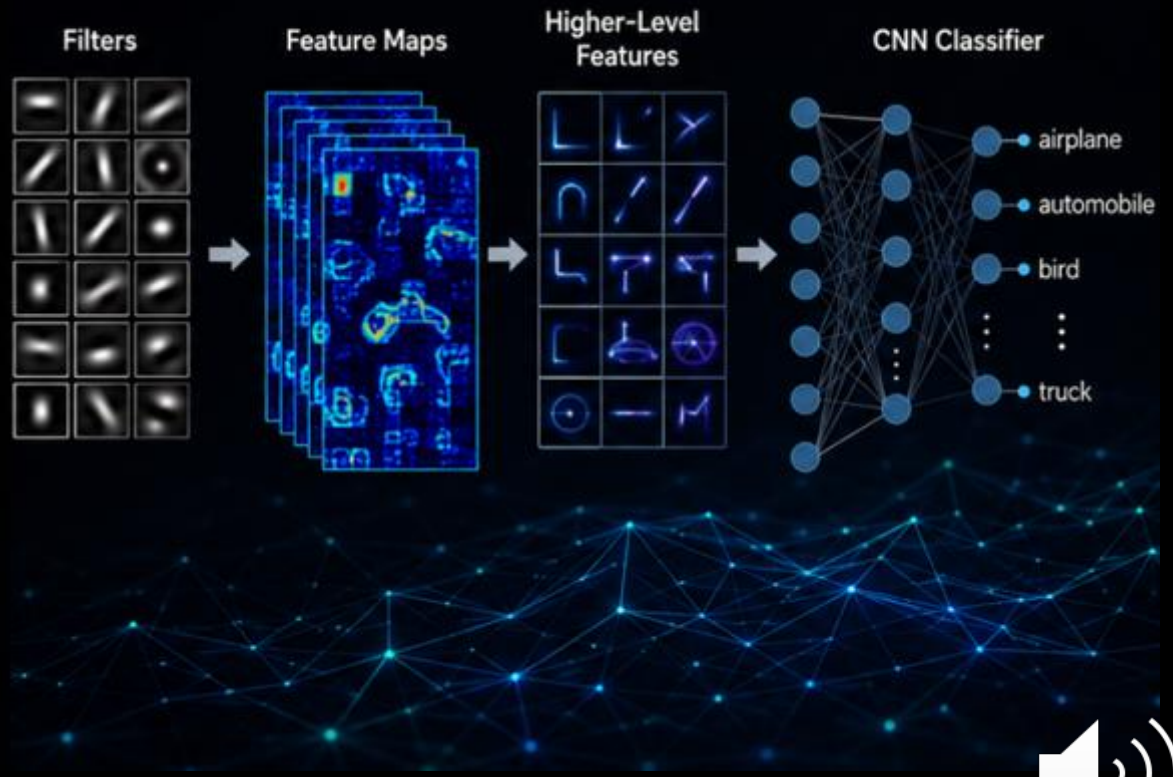
From Handcrafted Features to CNN

CIFAR-10 Image Classification

FCNN → SIFT → Dense SIFT → CNN

> Understanding how feature extraction changes model performance





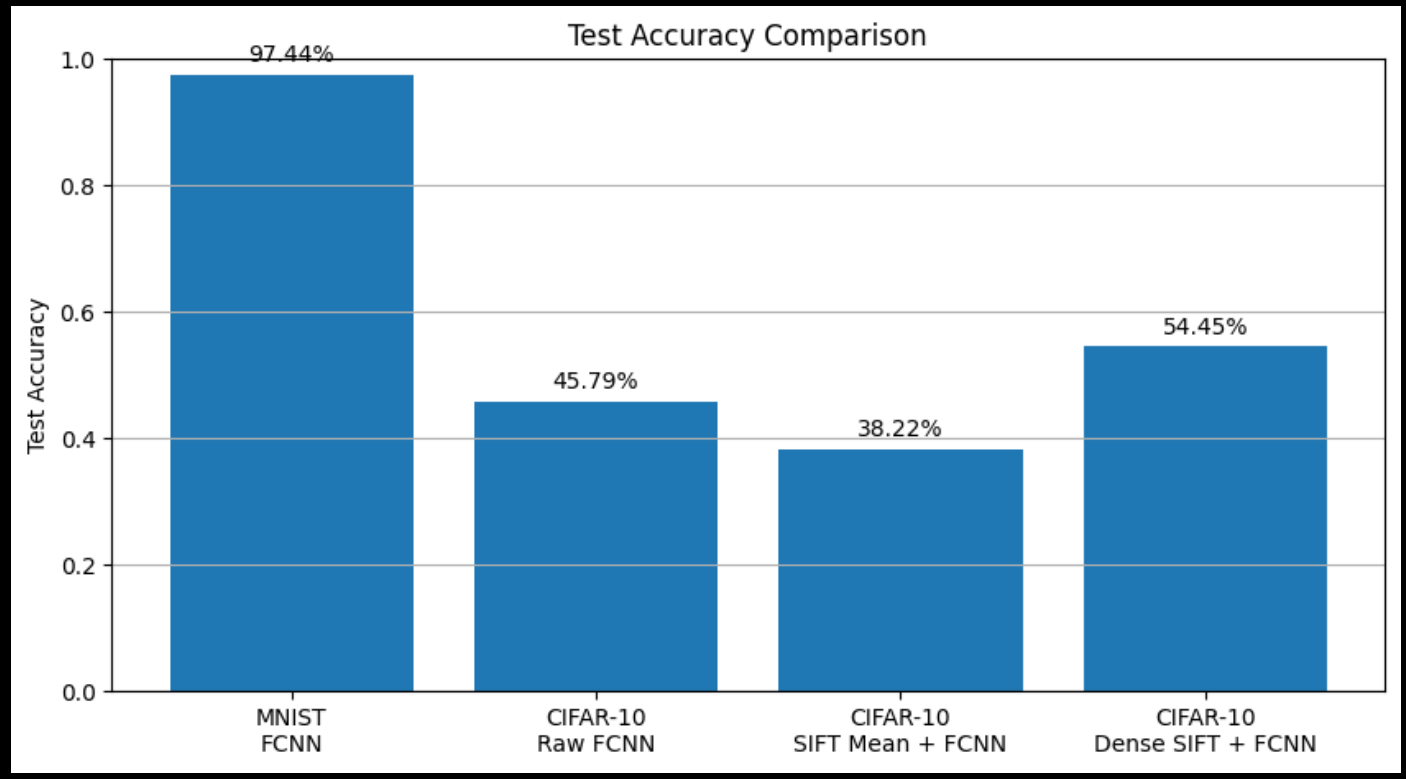
Maya Nazan Tem

Chapter-5
Special Topics - DATA-5900-98
Tennessee State University - Summer 2026

Previous Experiments: Feature Representation Matters

Before CNN, we tested four approaches:

- FCNN worked very well on MNIST.
- However, CIFAR-10 was harder because images are colorful, small, and visually more complex.
- SIFT + FCNN decreased performance because simple mean pooling lost spatial and color information.
- Dense SIFT + FCNN improved the result by extracting stronger handcrafted features.



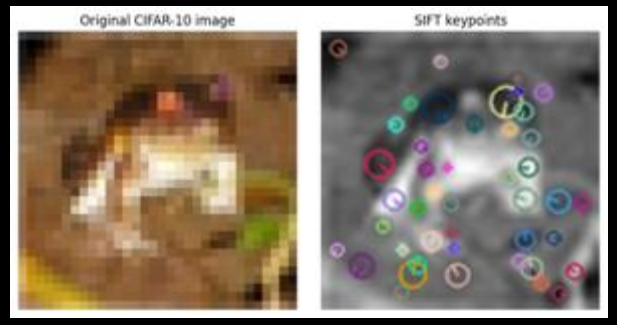
From SIFT to Dense SIFT: What Changed?

Our first SIFT + FCNN model used:

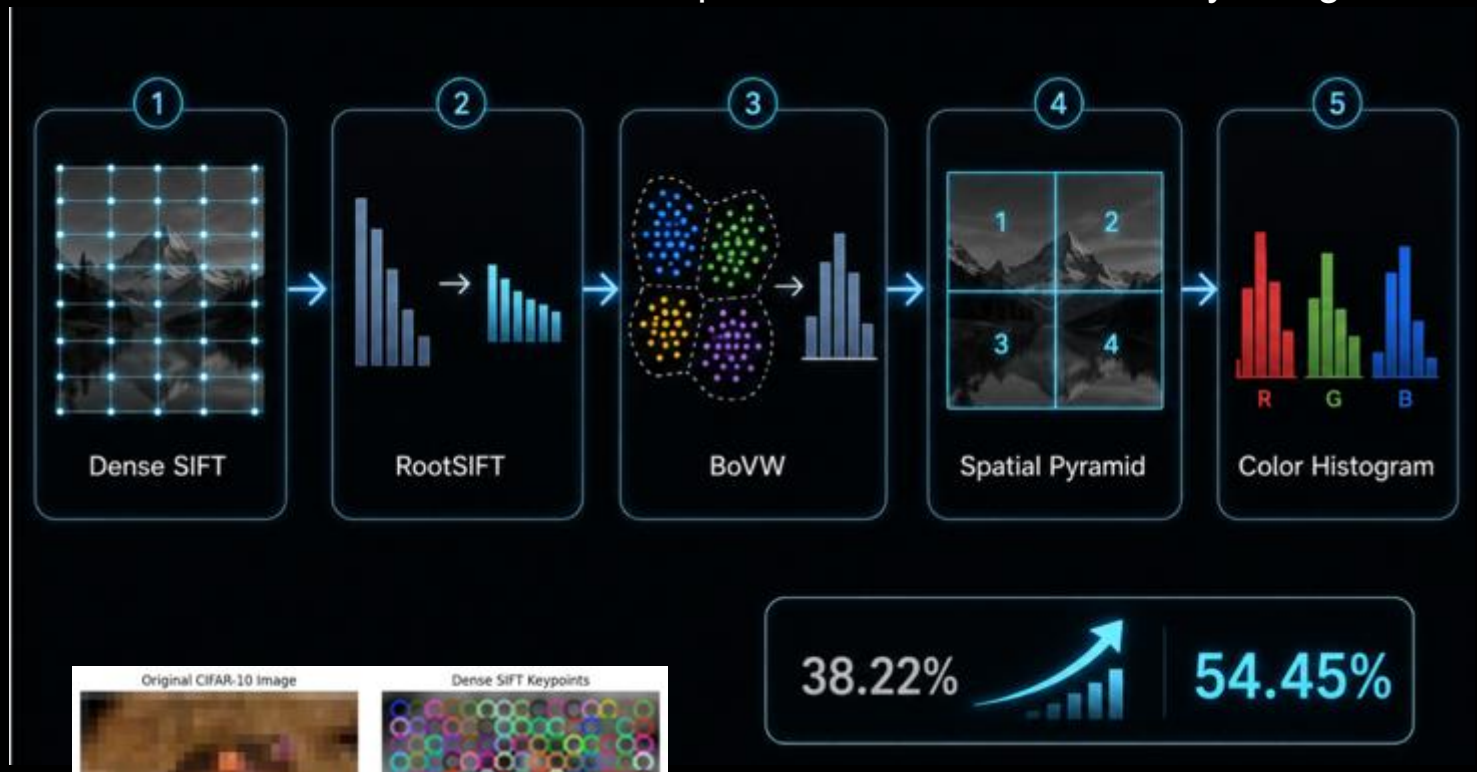
- standard SIFT keypoints
- mean pooling
- one 128-dimensional feature vector per image

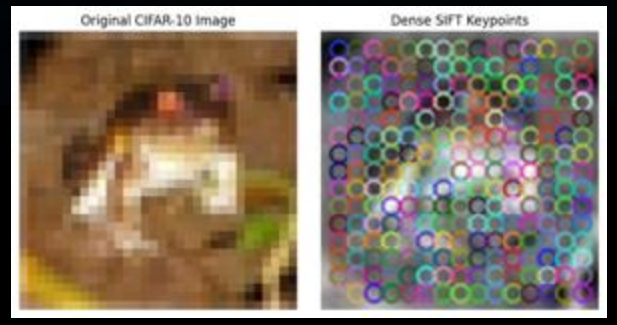
This approach was limited because it lost:

- spatial information
- color information
- many local image details



In Dense SIFT + FCNN, we improved feature extraction by using:





38.22% → 54.45%

Why Move to CNN?

Dense SIFT improved the result, but it still relied on handcrafted features.
This means that feature extraction was still manually designed.

CNN uses a different approach:
instead of manually designing features, it learns useful image features directly from data.

CNN learns features automatically through convolutional filters.

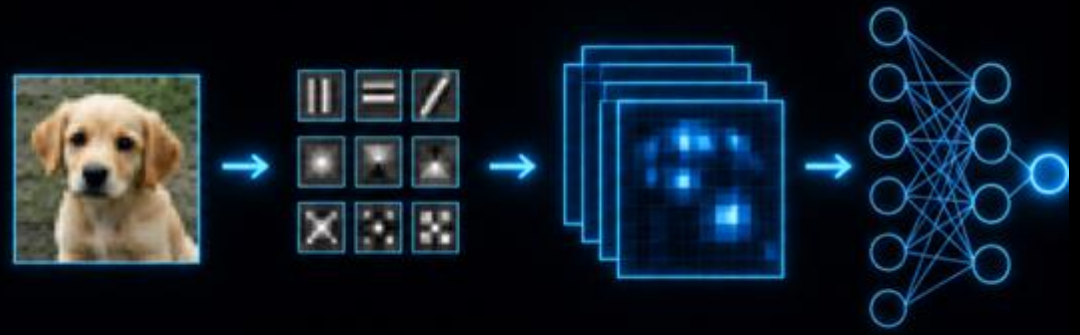
What is a CNN?

A Convolutional Neural Network is a neural network designed for image data.

Instead of treating an image as one long vector, CNN keeps the spatial structure of the image.

It learns patterns step by step:

edges → textures → shapes → object parts

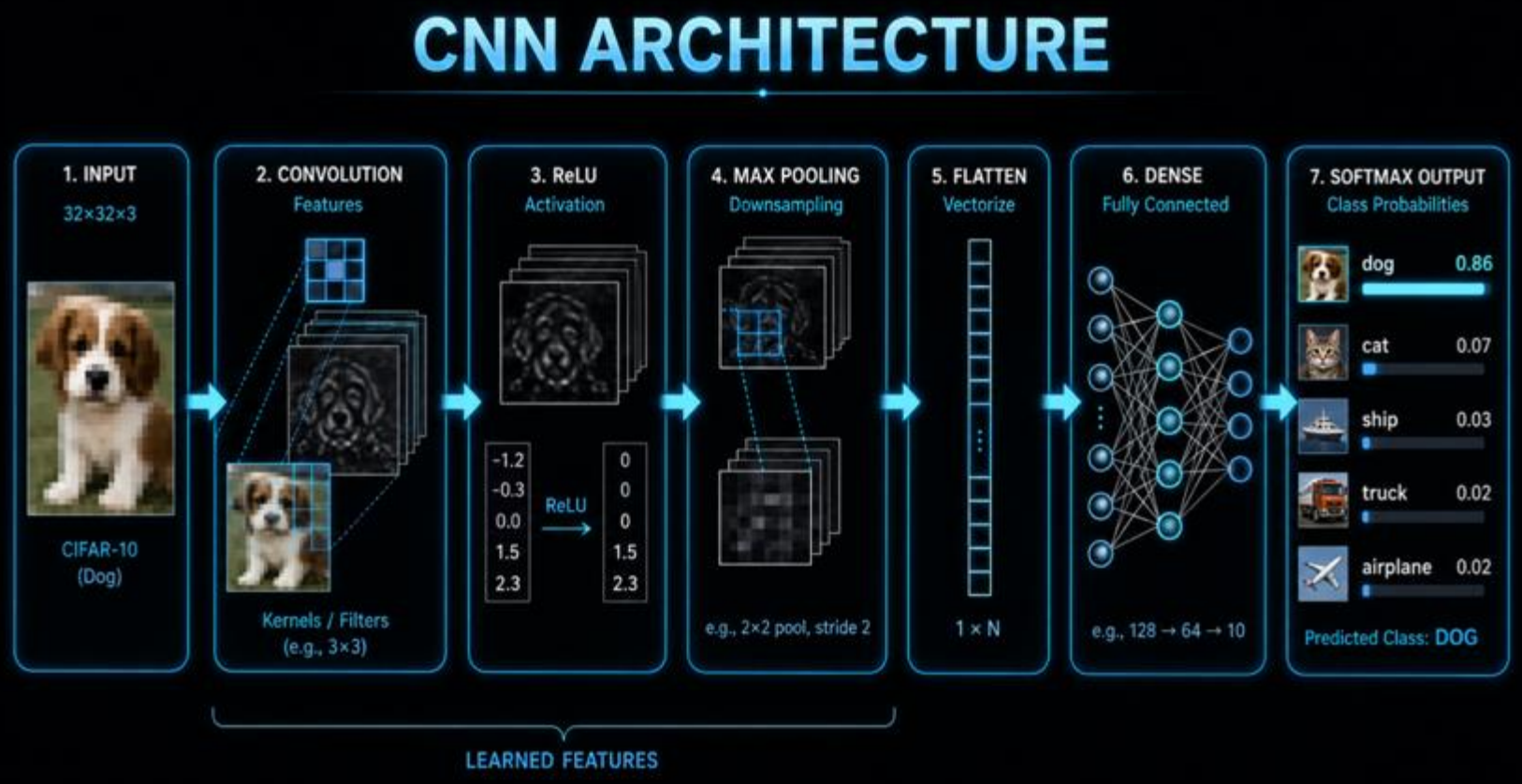


CNN learns visual features directly from images.

CNN Architecture

A CNN model usually has two main parts:

1. Feature Extraction: Convolution + ReLU + Pooling layers learn visual patterns.
2. Classification: Flatten + Dense + Softmax layers use these features to predict the class.



Input image → Convolution → ReLU → Pooling → Flatten → Dense → Output

Convolution and Kernels

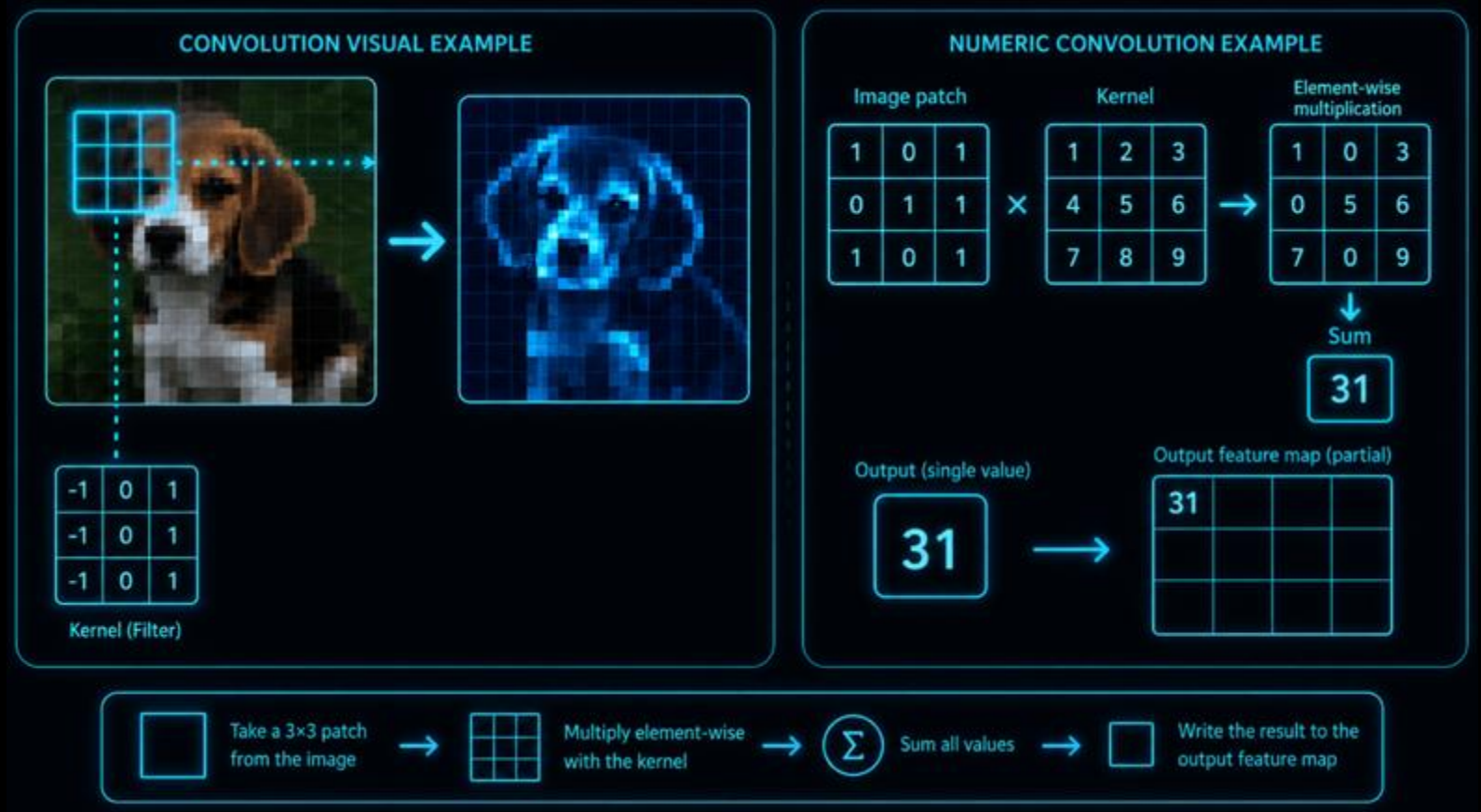
The convolution layer is the main feature extraction part of CNN.

It uses small filters, called kernels, to scan local regions of the image.

At each position, the kernel performs element-wise multiplication and then sums the results.

The output is a feature map, which shows where a pattern appears in the image.

Kernel values are learned and updated during training.



A kernel learns what visual pattern to detect.

Stride and Padding

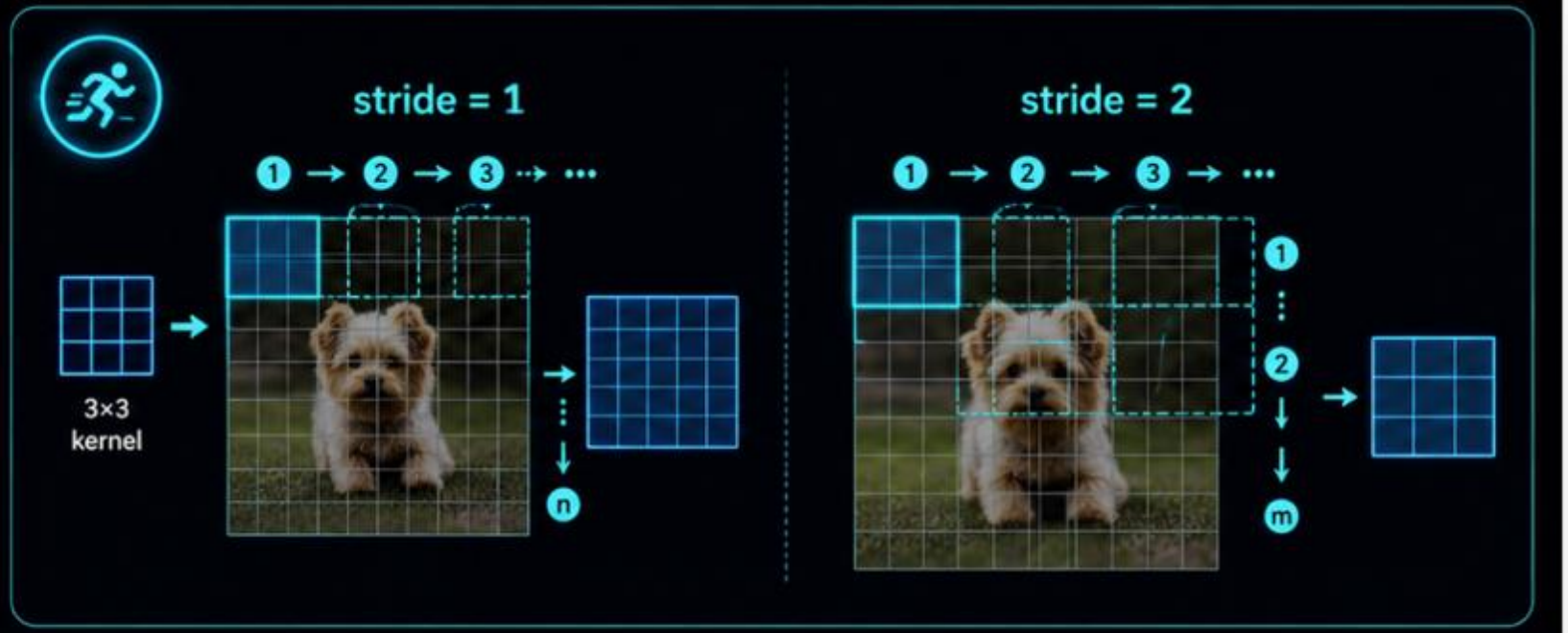
Stride controls how many pixels the kernel moves at each step.

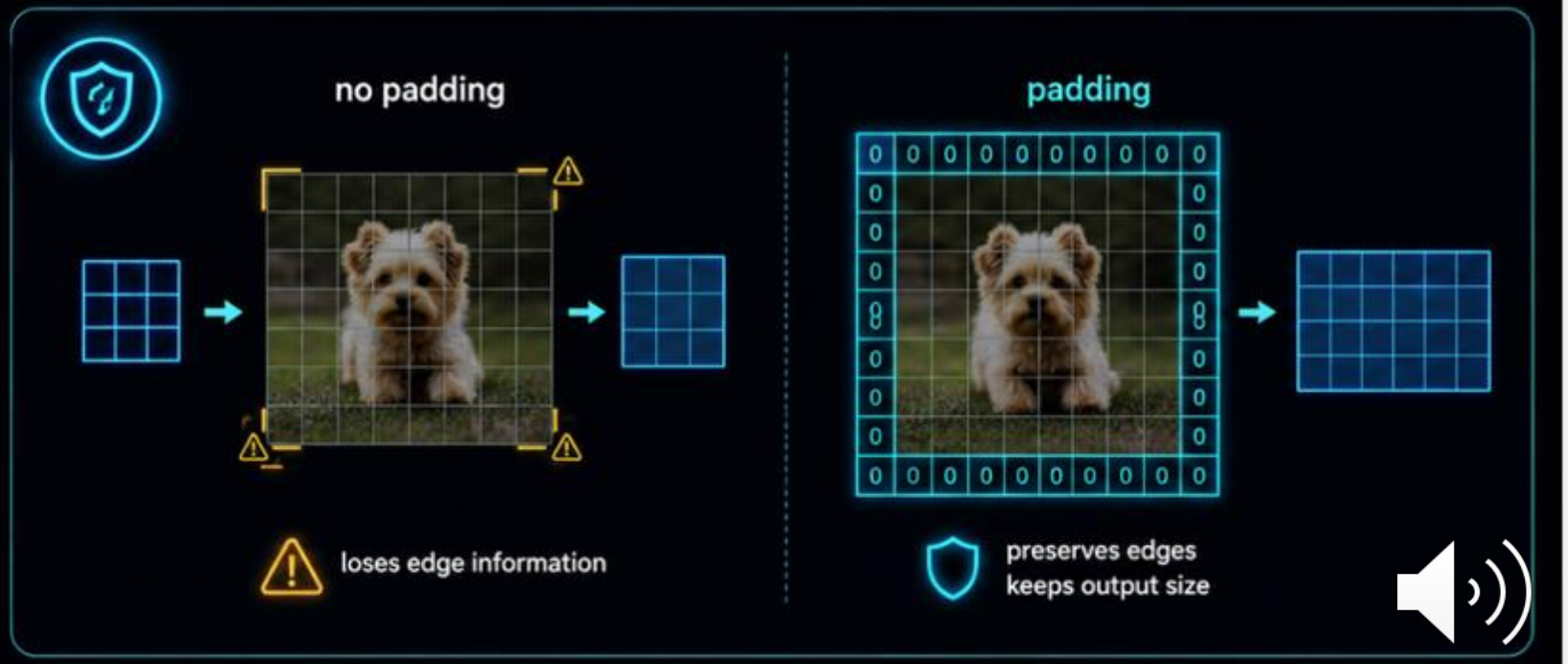
A larger stride makes the feature map smaller.

Padding adds extra pixels around the image.

It helps preserve edge information and controls the output size.

Stride controls movement.
Padding protects borders.





Activation Function: ReLU

After convolution, CNN applies an activation function.

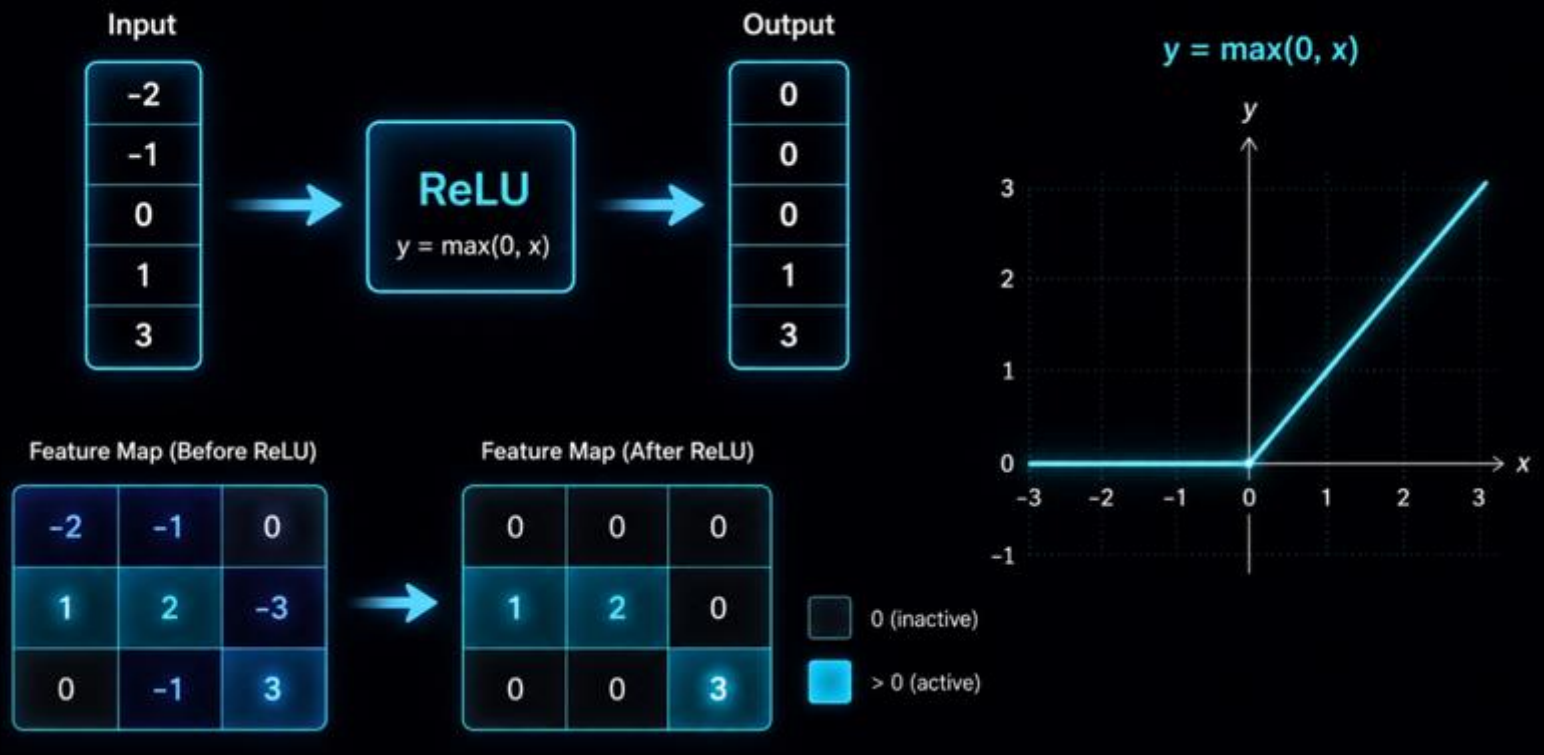
ReLU means Rectified Linear Unit.

It keeps positive values unchanged and changes negative values to zero.

Formula: f(x) = max(0, x)

This adds non-linearity and helps the model

learn more complex visual patterns.



ReLU activates useful features and removes negative responses.

Pooling Layer

Pooling reduces the size of the feature map.

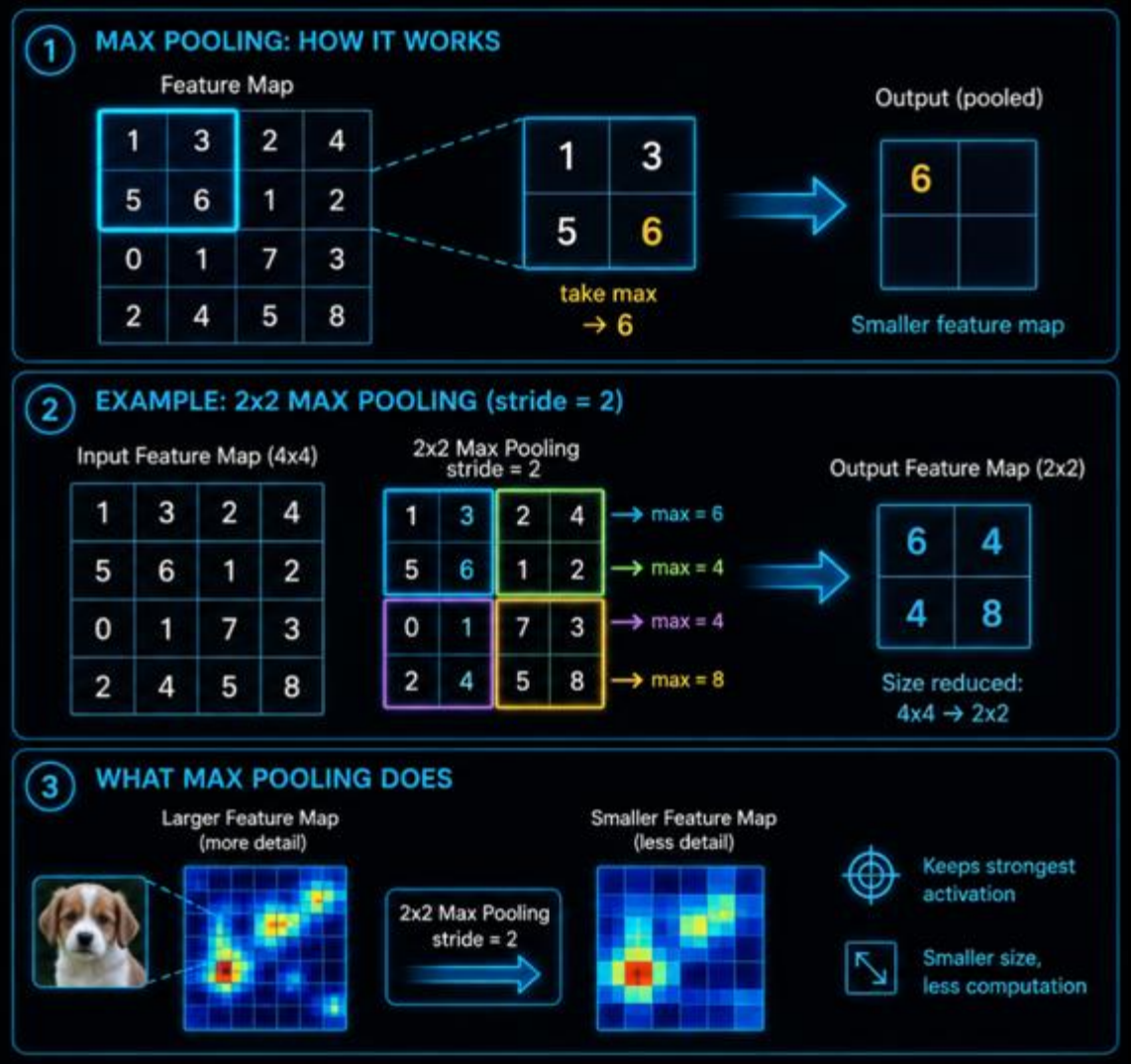
This helps decrease computation and makes the model more efficient.

The most common type is Max Pooling.

It selects the largest value from a small region.

This keeps the strongest feature while reducing the spatial size.

Pooling keeps important information and reduces dimension.



Example: 2×2 Max Pooling with stride 2

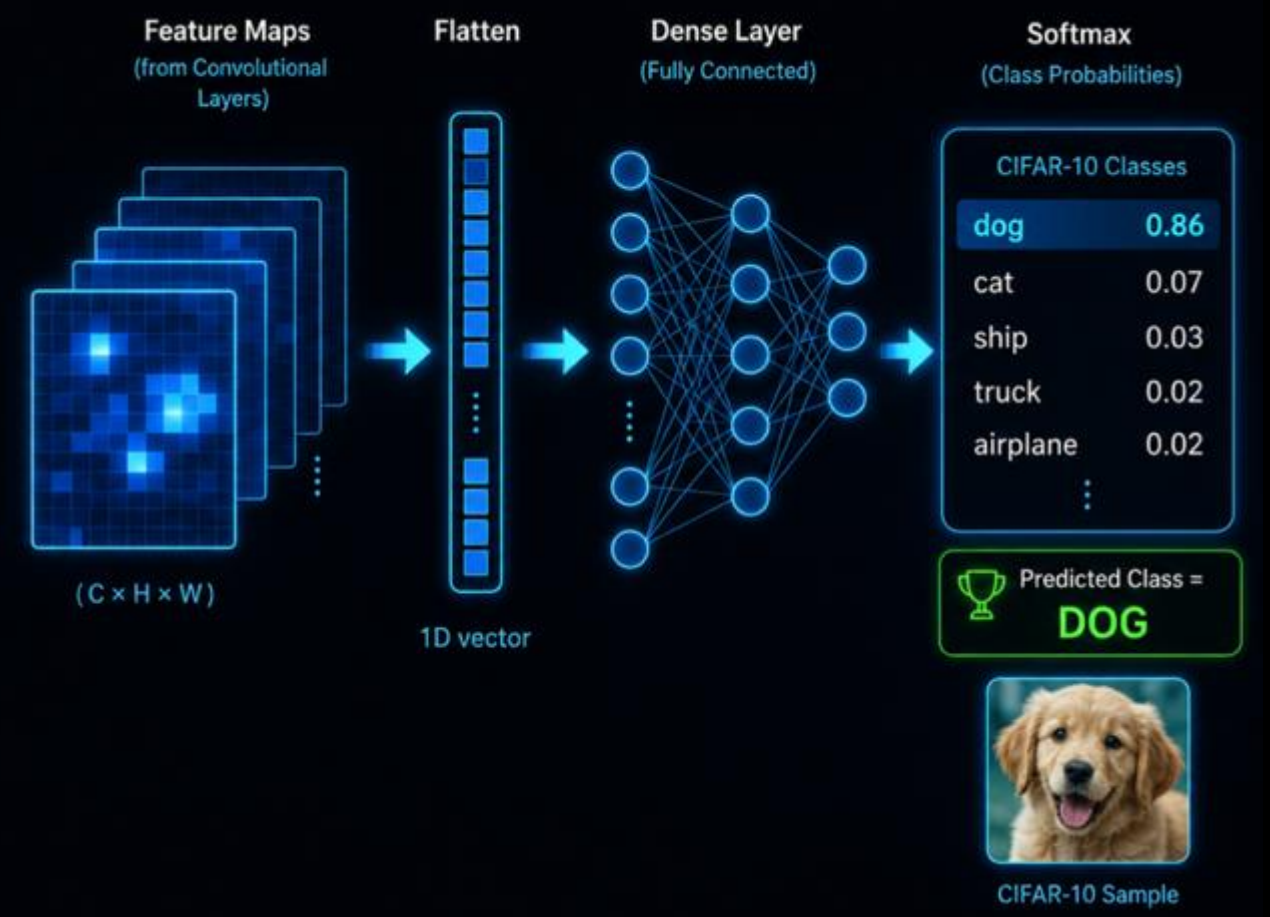
Flatten, Dense Layer and Softmax

After feature extraction, CNN prepares the learned features for classification.

Flatten converts feature maps into a one-dimensional vector.

Dense layers use this vector to combine features and predict the class.

Softmax converts the final outputs into class probabilities.



Feature maps become class predictions.

How CNN Learns

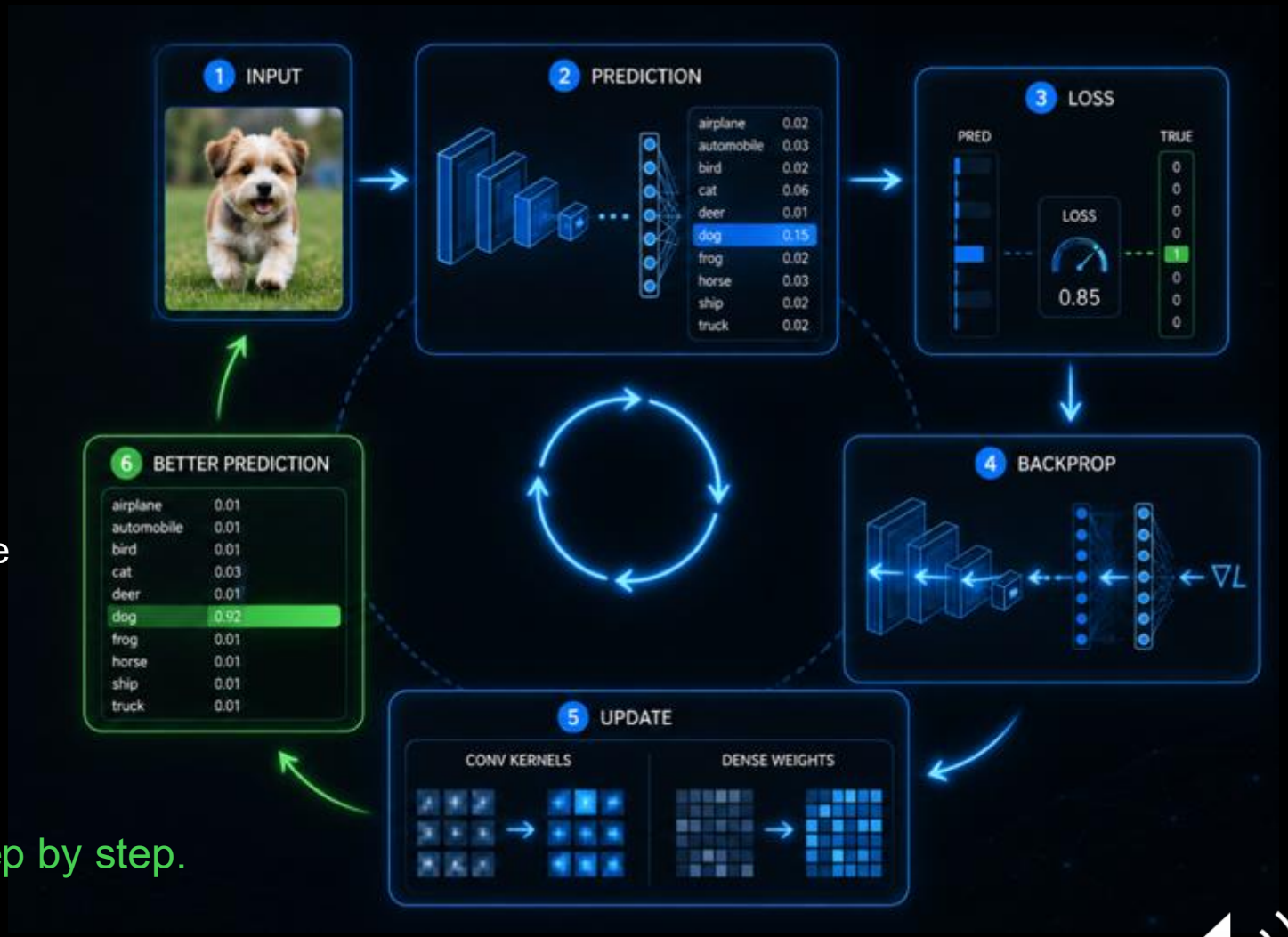
CNN first makes a prediction.

Then, the loss function measures how far the prediction is from the true label.

Backpropagation calculates how each weight contributed to the error.

The optimizer updates the weights, including the convolution kernels.

This process repeats during training.



CNN learns by reducing prediction error step by step.

Mathematical Foundations of CNN

CNN is based on repeated mathematical operations.

Weights and biases are learnable parameters.

Convolution: local patch × kernel → sum → feature map

Activation functions, such as ReLU, add non-linearity to the model.

ReLU: f(x) = max(0, x)

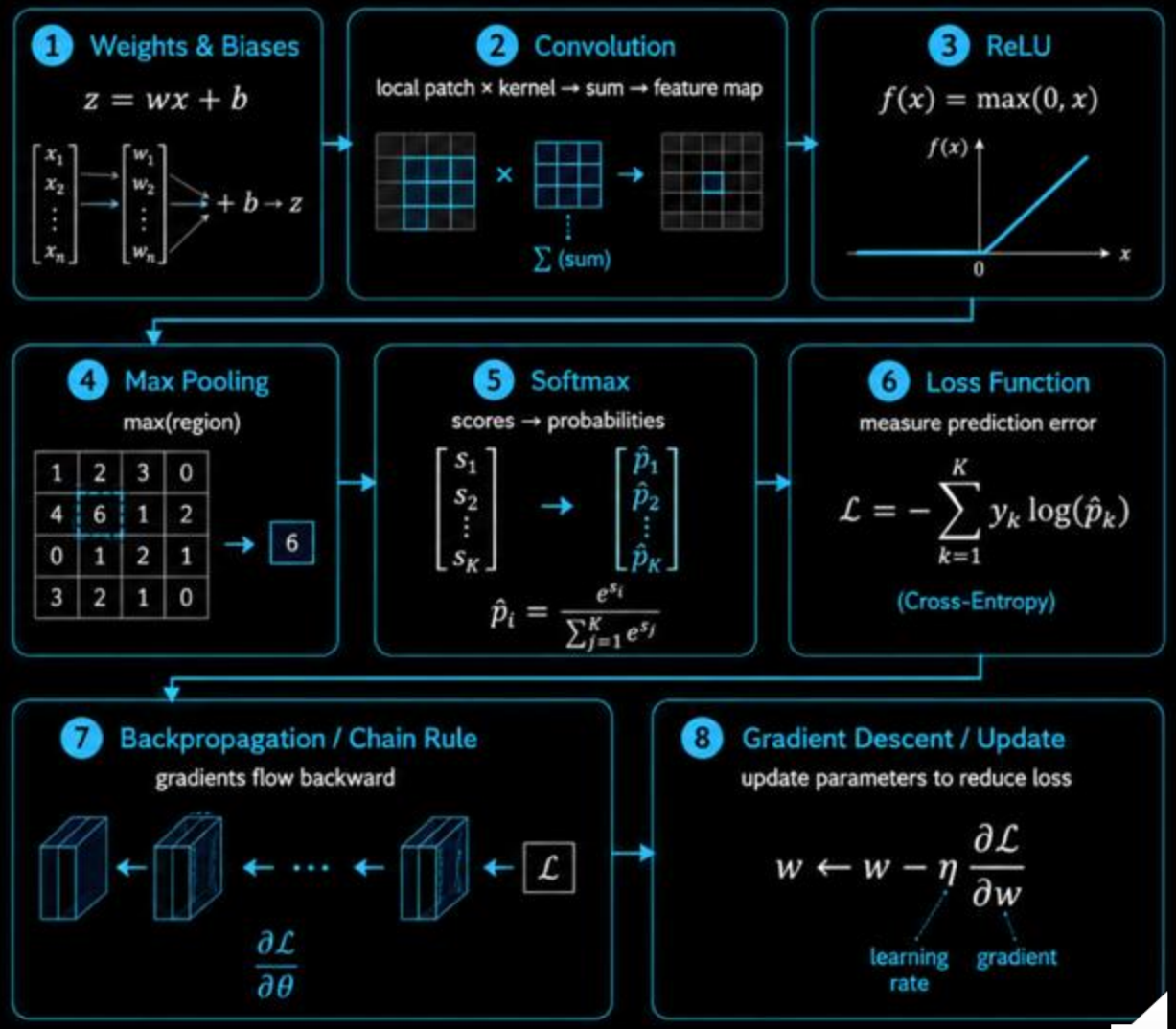
Max Pooling: keeps the maximum value from a small region

Softmax: converts final scores into class probabilities

The loss function measures prediction error.

Backpropagation uses the chain rule

Gradient descent updates weights and kernels to reduce the loss.



CNN learns by using gradients to update its parameters.

Our CIFAR-10 CNN Model

We used CIFAR-10 as the image classification dataset.

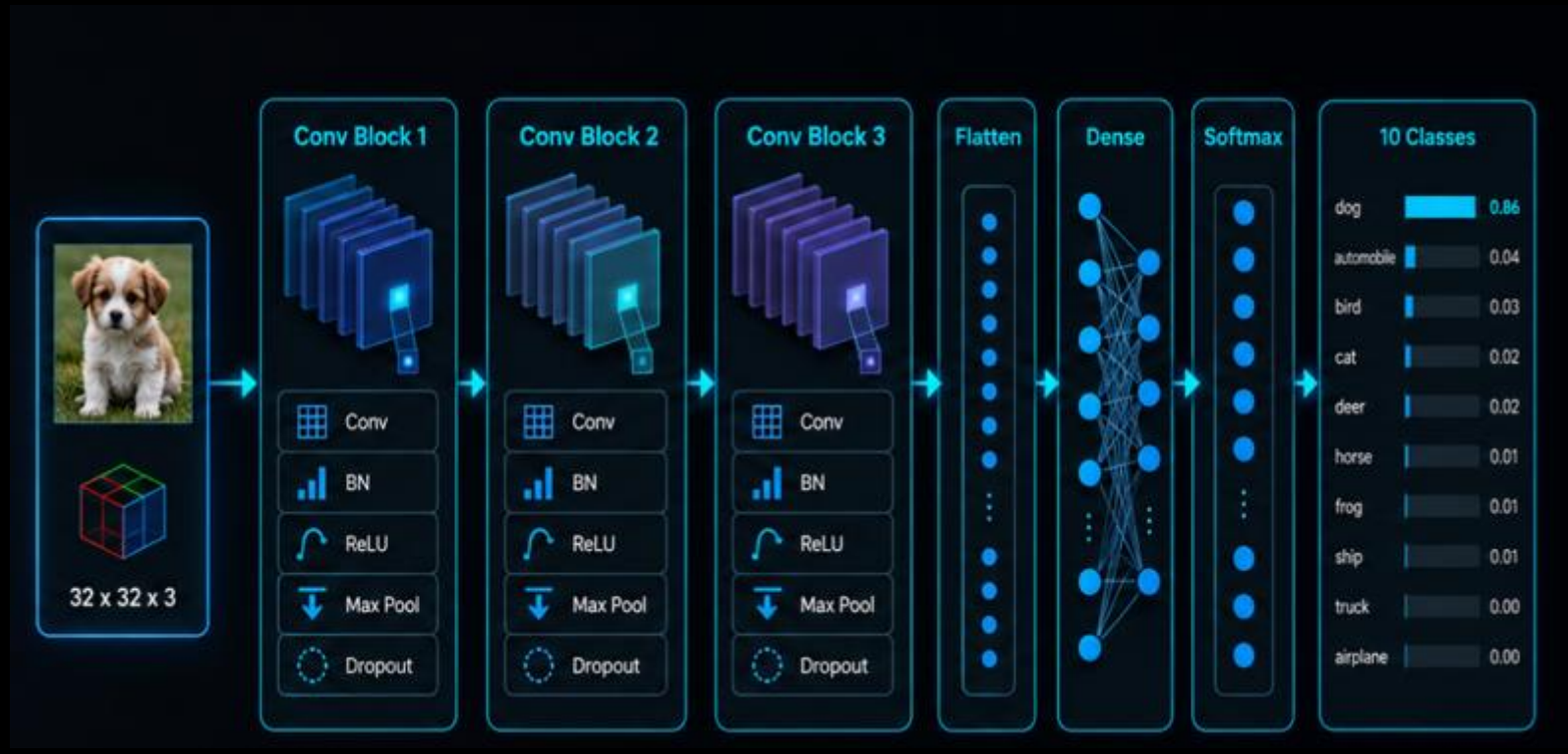
Input images have size 32 × 32 × 3.

The model uses convolution blocks to learn visual features from images.

Each block includes:

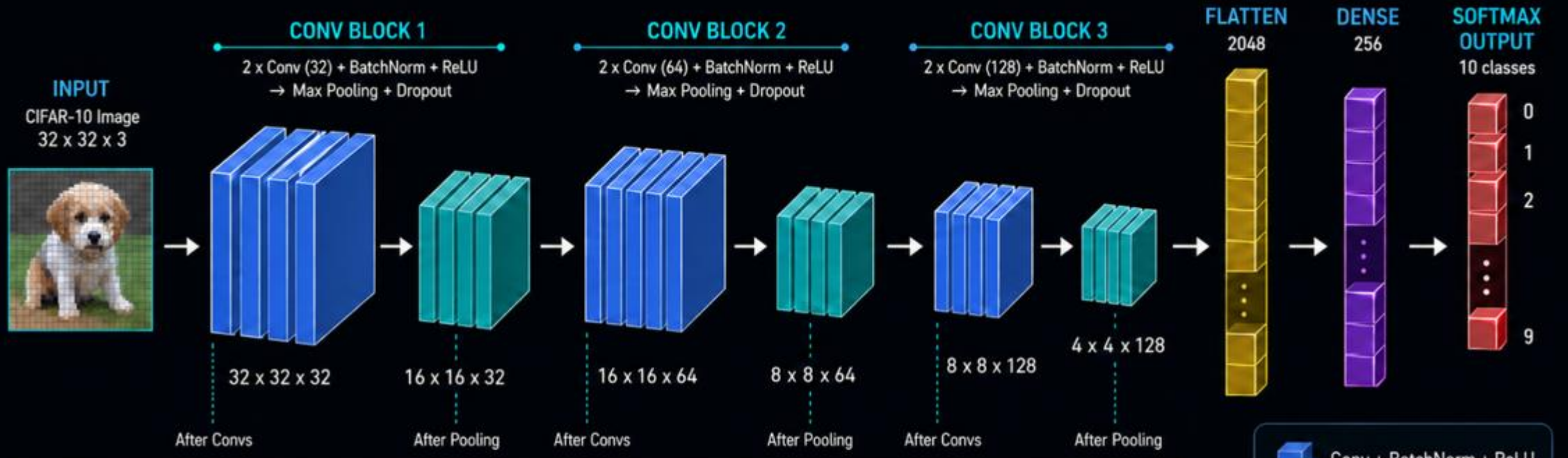
Convolution → Batch Normalization → ReLU → Max Pooling → Dropout

After feature extraction, the model uses Flatten, Dense, and Softmax layers for classification.



The CNN learns image features directly from CIFAR-10 images.

Detailed CNN Architecture



Conv Block 1
- Conv2D (32, 3x3, padding=1)
- BatchNorm
- ReLU
- Conv2D (32, 3x3, padding=1)
- BatchNorm
- ReLU
- Max Pooling (2x2)
- Dropout (p=0.25)

Conv Block 2
- Conv2D (64, 3x3, padding=1)
- BatchNorm
- ReLU
- Conv2D (64, 3x3, padding=1)
- BatchNorm
- ReLU
- Max Pooling (2x2)
- Dropout (p=0.25)

Conv Block 3
- Conv2D (128, 3x3, padding=1)
- BatchNorm
- ReLU
- Conv2D (128, 3x3, padding=1)
- BatchNorm
- ReLU
- Max Pooling (2x2)
- Dropout (p=0.25)

Flatten
- Flatten to vector (2048)

Dense
- Dense (256)
- ReLU
- Dropout (p=0.5)

Output
- Dense (10)
- Softmax

Conv + BatchNorm + ReLU
Max Pooling
Flatten
Dense / Fully Connected
Softmax Output

The CNN gradually reduces spatial size while learning deeper feature representations.

Model Summary and Training Setup

Model Summary

- Input: 32 × 32 × 3
- 3 convolution blocks: 32, 64, 128 filters
- Flatten: 2048
- Dense: 256
- Output: 10 classes
- Total parameters: 816,938

Training Setup

- Optimizer: Adam
- Loss: sparse_categorical_crossentropy
- Metric: accuracy
- Validation split: 0.2
- Epochs: 35
- Batch size: 64
- Callbacks: EarlyStopping, ReduceLROnPlateau

Layer (type)	Output Shape	Param #
data_augmentation (Sequential)	(None, 32, 32, 3)	0
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
batch_normalization_1 (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_2 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
batch_normalization_3 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
conv2d_4 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_4 (BatchNormalization)	(None, 8, 8, 128)	512
conv2d_5 (Conv2D)	(None, 8, 8, 128)	147,584
batch_normalization_5 (BatchNormalization)	(None, 8, 8, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
dropout_2 (Dropout)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 256)	524,544
batch_normalization_6 (BatchNormalization)	(None, 256)	1,024
dropout_3 (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 10)	2,570

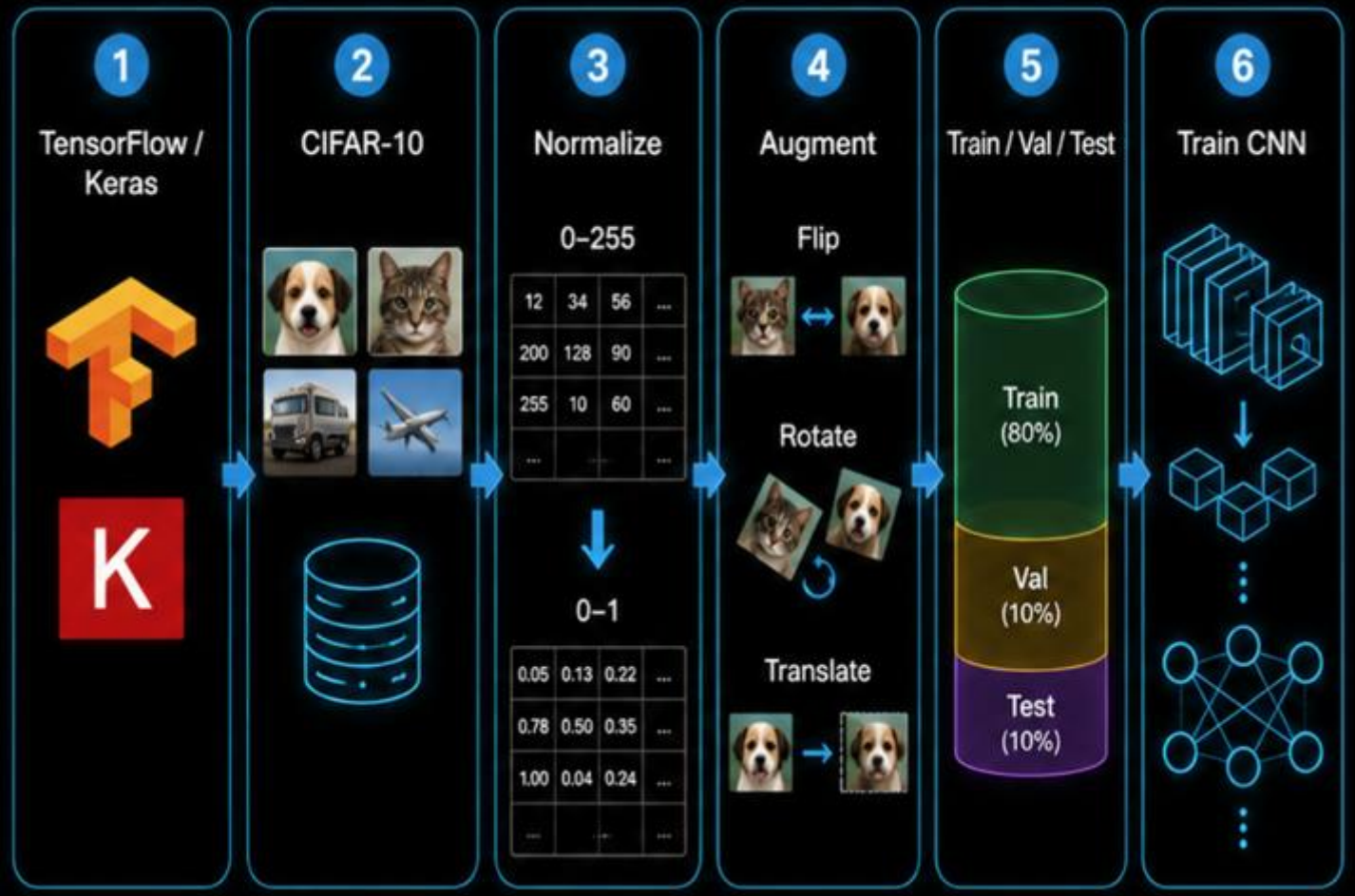
Total params: 816,938 (3.12 MB)
Trainable params: 815,530 (3.11 MB)
Non-trainable params: 1,408 (5.50 KB)

The model summary shows the CNN structure, and the training setup defines how the model learns.

Implementation Workflow

We used TensorFlow/Keras to build the CNN model.

- Dataset: CIFAR-10 images and labels were loaded from Keras.
- Preprocessing: Pixel values were normalized from 0–255 to 0–1.
- Data Augmentation: Random flip, translation, and rotation were used to reduce overfitting.
- Data Split: Training data was divided into training and validation sets. The test set was kept unseen for final evaluation.



Before training, the data must be prepared correctly.

Training Configuration

The model was compiled before training.

- Optimizer: Adam updates weights efficiently during training.
- Loss: sparse_categorical_crossentropy is suitable for multi-class classification with integer labels.
- Metric: accuracy measures the percentage of correct predictions.
- Callbacks: EarlyStopping prevents overfitting.
- ReduceLROnPlateau lowers the learning rate when validation performance stops improving.

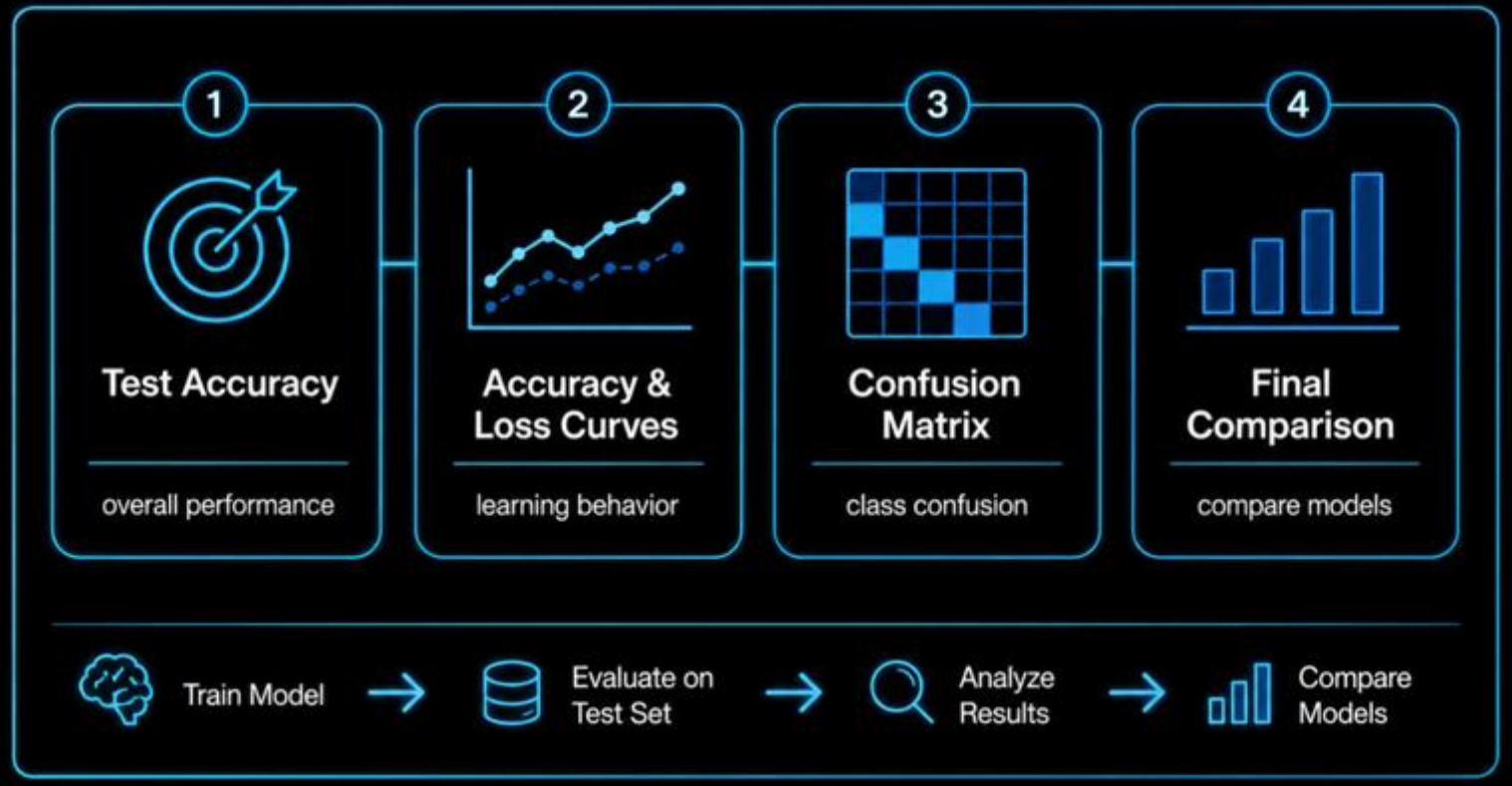
Training settings control how the model learns from data.

Evaluation Plan

After training, we evaluate the CNN on the unseen test set.

I use four evaluation steps:

- Test accuracy to measure overall performance.
- Accuracy and loss curves to analyze learning behavior.
- Confusion matrix to identify confused classes.
- Final comparison to compare CNN with previous models.



Evaluation shows both model performance and model mistakes.

Training Results

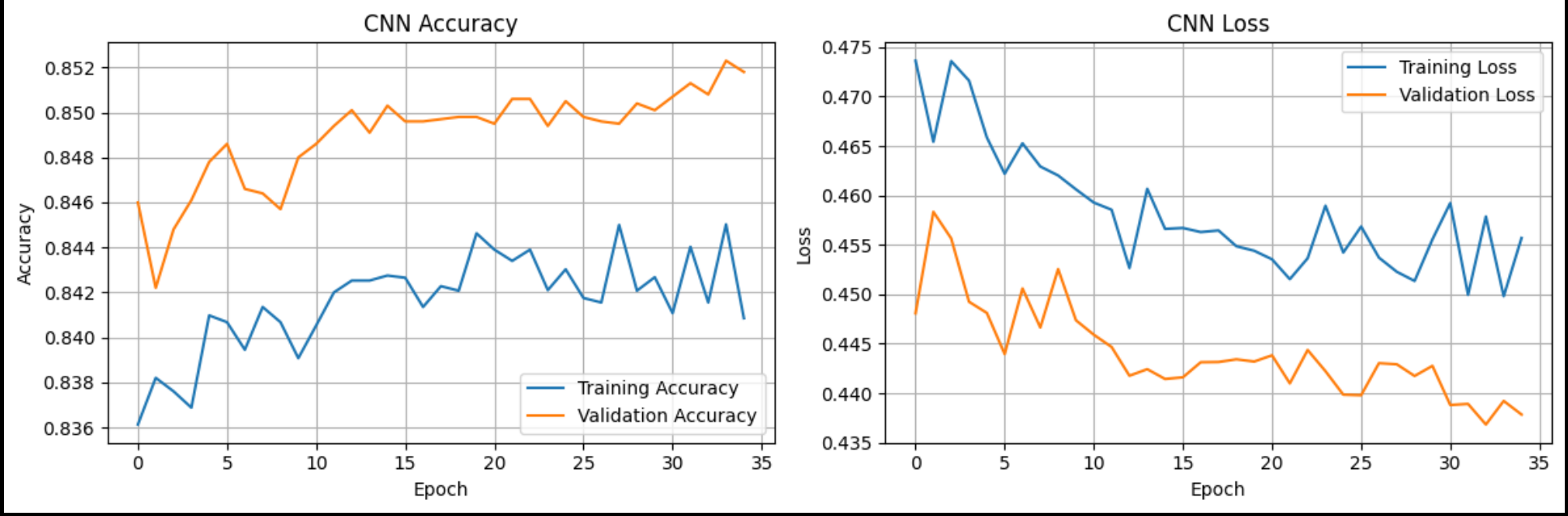
The CNN reached stable performance during training.

Training and validation accuracy increased over epochs.

Validation accuracy stayed around 85%.

Loss values generally decreased during training.

The curves do not show severe overfitting.



The CNN learned stable visual representations during training.

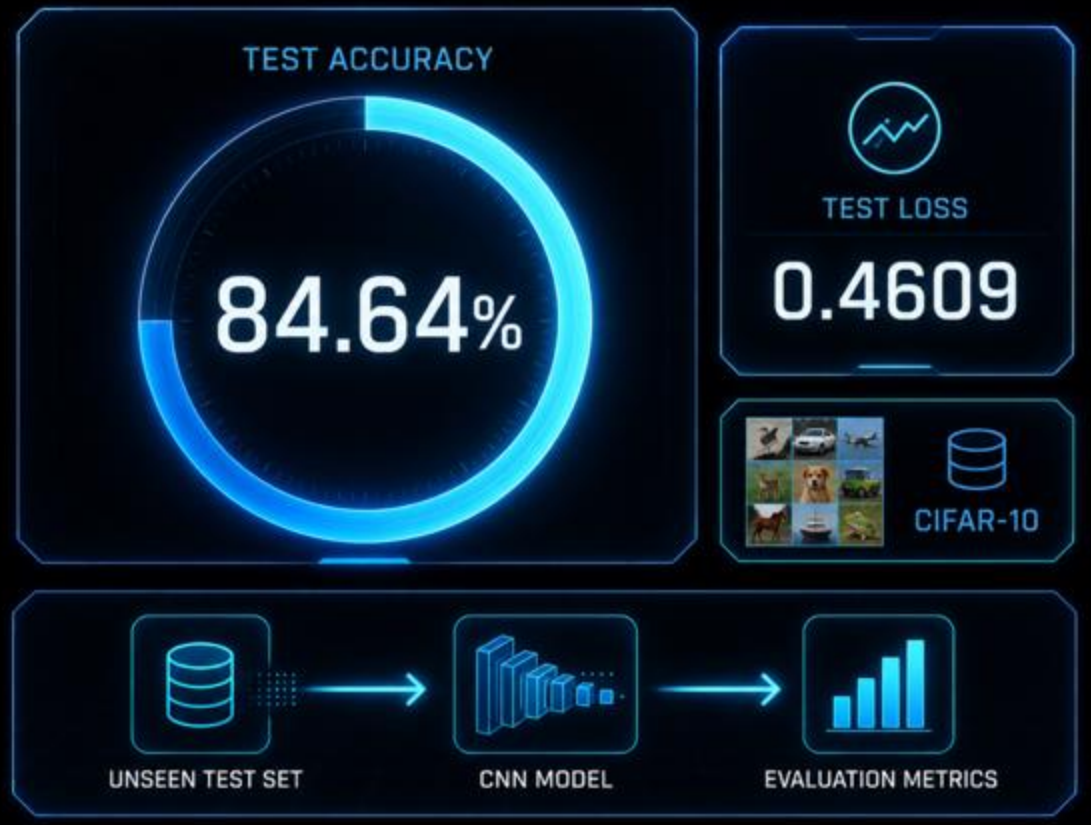
Test Set Evaluation

After training, we evaluated the CNN on the unseen test set.

The test set was not used during training or validation.

We measured:

- Test Loss: 0.4609
- Test Accuracy: 84.64%
- These values show how well the model generalizes to new CIFAR-10 images.



The CNN achieved 84.64% test accuracy on CIFAR-10.

Confusion Matrix

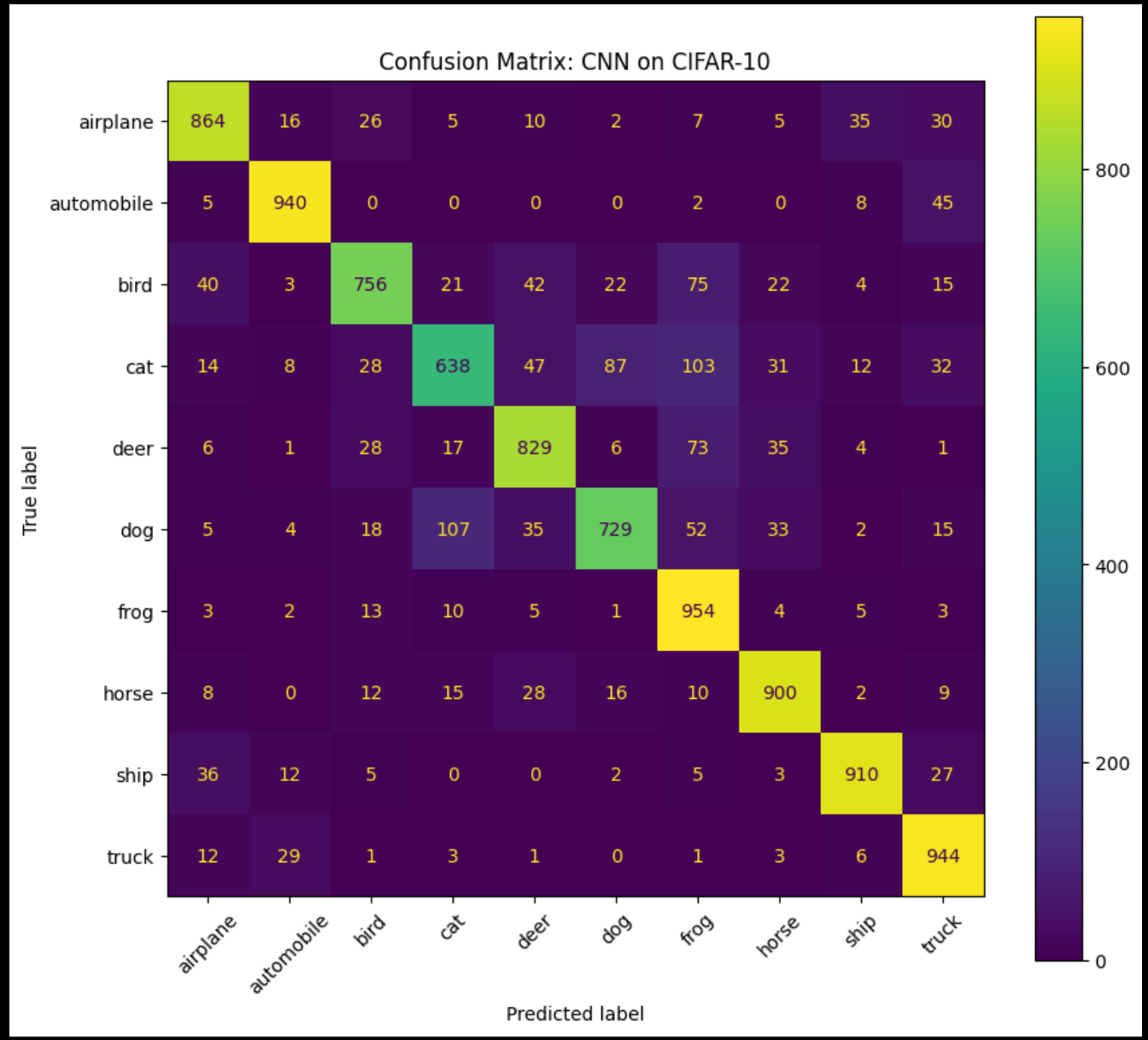
The confusion matrix shows class-level performance.

Most correct predictions appear on the diagonal.

The model performed strongly on: automobile, frog, truck, ship, and horse.

The most difficult classes were: cat, dog, and bird.

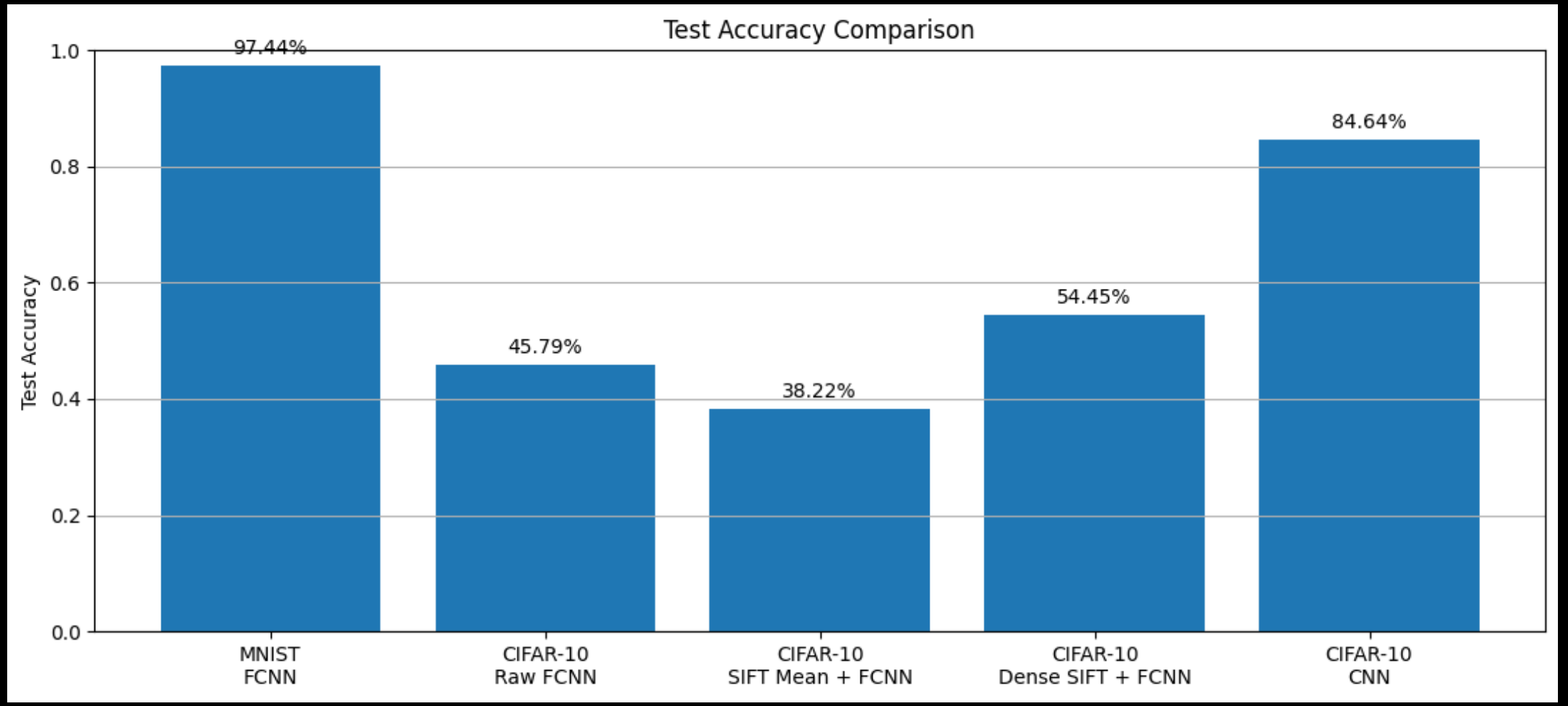
Some visually similar classes were confused.



The model performs well overall, but animal classes are harder to separat

Final Model Comparison

It reached 84.64% test accuracy by learning visual features automatically.



CNN clearly outperformed handcrafted feature approaches on CIFAR-10.

Conclusion

CIFAR-10 progression with a separate MNIST reference

MNIST REFERENCE

MNIST FCNN

Accuracy

97.44%

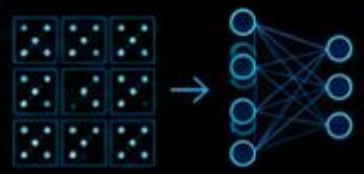
Different dataset
Not directly comparable

1 Raw FCNN

Accuracy (CIFAR-10)

45.79%

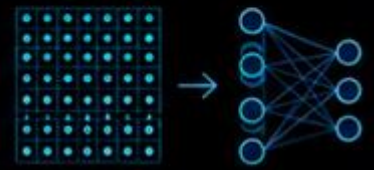
2 SIFT + FCNN



Accuracy (CIFAR-10)

38.22%

3 Dense SIFT + FCNN



Accuracy (CIFAR-10)

54.45%

4 CNN

Best CIFAR-10 Result

Accuracy (CIFAR-10)

On CIFAR-10, **CNN** achieved the **strongest performance**.
MNIST FCNN is shown only as a separate reference because it uses a different dataset.

References

CNN_CIFAR10

https://colab.research.google.com/drive/1VnmoEm-CaOoCEvuonqLWyLXoXXoPfRnF#scrollTo=foBToUvwvcvS

DENSE_SIFT_CIFAR10

https://colab.research.google.com/drive/136fll4q3rWhP6QX-YFM-nvathRUXnwf6#scrollTo=C7TDTabd_Jss